

# Reviewer Pack — FCA Antichain Width of Implicant Lattice Caps DNF-MCSP Withi...

Entry 6010af3a708c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 22:41:41 UTC

## FCA Antichain Width of Implicant Lattice Caps DNF-MCSP Within Factor $2n$

Entry ID: 6010af3a708c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 22:41:41 UTC

### 1. Conjecture

**Field A** (mathematical branch): Formal Concept Analysis / Galois lattices (Birkhoff 1940; Ganter-Wille 1999 ‘Formal Concept Analysis’; Davey-Priestley ‘Introduction to Lattices and Order’). The bridge to meta-complexity has essentially zero presence: an arXiv search ‘formal concept analysis’ AND (‘MCSP’ OR ‘minimum DNF’ OR ‘Kolmogorov complexity’ OR ‘meta-complexity’) returns 0 direct hits with <5 adjacent papers (all on data-mining association rules and Pawlak rough sets, never on DNF-MCSP or  $K^t$  gaps). Distinct from blacklisted Mahler-measure (a single complex-analytic functional on the truth-table polynomial) — here the invariant is the antichain WIDTH of a CATEGORICAL Galois lattice on the implicant context, a relational order-theoretic functor of the (assignments  $\times$  literals) bipartite incidence, not a single truth-table functional, sidestepping the trap (j). **Field B** (complexity object): DNF-MCSP / minimum-DNF size  $\sigma_{\text{DNF}}(f)$  at small  $n$ , the simplest meta-complexity object in the Hirahara-Oliveira / Allender-Hellerstein-McCabe-Pitassi-Saks / Ko 1991 / Masek 1979 NP-completeness tradition; equivalent by Quine-McCluskey to Set Cover over prime implicants and the target of Umans 1999’s  $n^{1-\varepsilon}$  hardness-of-approximation regime.

#### Statement:

For every Boolean function  $f: \{0,1\}^n \rightarrow \{0,1\}$  with  $n \leq 5$ , let  $K_f := (M_f^+, L_n, I)$  be the implicant context with objects  $M_f^+ = \{x : f(x)=1\}$ , attributes  $L_n =$  the  $2n$  signed literals over  $(x_1, \dots, x_n)$ , and incidence  $I(x, \ell) := 1$  iff  $x$  satisfies  $\ell$ . Let  $w(f)$  be the antichain width (size of the largest set of pairwise-incomparable formal concepts) of the concept lattice  $L(K_f)$ . Then  $\sigma_{\text{DNF}}(f) \leq w(f) \leq 2n \cdot \sigma_{\text{DNF}}(f)$ . A single tested  $f$  with  $w(f) < \sigma_{\text{DNF}}(f)$  refutes the lower bound; a single tested  $f$  with  $w(f) > 2n \cdot \sigma_{\text{DNF}}(f)$  refutes the upper bound.

#### Rationale (proposer’s reasoning):

Prime implicants of  $f$  are pairwise-incomparable formal concepts (no PI’s literal set properly contains another’s by primality), so they form an antichain in  $L(K_f)$  of size  $\#PI(f) \geq \sigma_{\text{DNF}}(f)$  — the lower direction is structural and serves as a sanity check. The conjectured upper bound  $w(f) \leq 2n \cdot \sigma_{\text{DNF}}(f)$  says the FCA antichain spread of the implicant context is sandwiched against the min-cover size up to a literal-count factor, asserting that the Galois-lattice structure of  $K_f$  is a  $2n$ -tight surrogate for the NP-hard DNF-min problem in line with Umans 1999’s  $\tilde{O}(n)$ -hardness-of-approximation regime. The bridge uses  $\subseteq$  and  $\cap$  on attribute sets (non-ring; no  $F_q$  polynomial extension preserves Galois

closure), so it is Aaronson–Wigderson algebrization-safe, and the inequality is a structural relation between two NP-flavored invariants rather than a P/poly natural property, so it does not trigger Razborov–Rudich (no claim about distinguishing P/poly from random is made — only a sandwich on two interrelated representation-side invariants).

**Taxonomy category:** META\_COMPLEXITY (status at proposal time: alive)

## 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 80e4dd022e373c4d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

### Acceptance criterion:

Across all 256 functions at  $n=3$ , all 65536 at  $n=4$ , and 30 seeds  $\times$  34 sampled truth-tables at  $n=5$ , every tested  $f$  must satisfy  $\sigma\_DNF(f) \leq w(f) \leq 2n \cdot \sigma\_DNF(f)$ . Report min/max of  $w/\sigma\_DNF$ ; a single violator on either side falsifies.

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL PROOFS | SAFE    | 0.99       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.95       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.95       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - formal concept analysis implicant lattice DNF minimization - concept lattice antichain width Boolean function complexity - Galois lattice prime implicants minimum DNF meta-complexity

**Top relevant hits considered:** - [<http://arxiv.org/abs/0710.4339v1>] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [<http://arxiv.org/abs/1111.0981v1>] Form factors for  $B$  to  $K\ell$  semileptonic decay from three-flavor lattice QCD - [<http://arxiv.org/abs/1210.2401v1>] Distributed Formal Concept Analysis Algorithms Based on an Iterative MapReduce Framework - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1308.1705v1>] Rotating lattice - [s2:10.3390/math11143047] Role Minimization Optimization Algorithm Based on Concept Lattice Factor - [s2:10.14569/ijacsa.2023.0140953] K-Means Extensions for Clustering Categorical Data on Concept Lattice

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90s$  wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from collections import defaultdict, deque
```

*# Helper functions for matrix operations*

```
def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m = len(A)
    n = len(A[0])
    augmented_matrix = [A[i] + [b[i]] for i in range(m)]

    for col in range(n):
        max_row = col
        for row in range(col+1, m):
            if abs(augmented_matrix[row][col]) > abs(augmented_matrix[max_row][col]):
                max_row = row

        augmented_matrix[col], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[col]

        pivot = augmented_matrix[col][col]
        for i in range(col, n+1):
            augmented_matrix[col][i] /= pivot

        for row in range(m):
            if row != col:
                factor = augmented_matrix[row][col]
                for i in range(col, n+1):
                    augmented_matrix[row][i] -= factor * augmented_matrix[col][i]

    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = augmented_matrix[i][-1]
        for j in range(i+1, n):
            x[i] -= augmented_matrix[i][j] * x[j]

    return x
```

*# Quine-McCluskey algorithm to find prime implicants*

```
def quine_mccluskey(minterms):
    minterms = sorted(set(minterms))
    variables = len(bin(max(minterms)))[2:]
    implicants = []
    for term in minterms:
        implicants.append([term, 1])

    while True:
        new_implicants = []
        for i in range(len(implicants)):
            for j in range(i+1, len(implicants)):
                # Check if terms differ by exactly one bit
```

```

        diff_count = sum(1 for a, b in zip(bin(implicants[i][0])[2:].zfill(variables), bin(
        if diff_count == 1:
            new_term = implicants[i][0] ^ (1 << [i for i, (a, b) in enumerate(zip(bin(imp
            new_implicants.append([new_term, implicants[i][1] + implicants[j][1]])
    if not new_implicants:
        break
    implicants = new_implicants

prime_implicants = [term for term, _ in implicants if all(bin(term)[2:].zfill(variables).count
return prime_implicants

# Function to compute the antichain width of a concept lattice
def antichain_width(concept_lattice):
    n = len(concept_lattice)
    adj_matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if all(concept_lattice[i][k] <= concept_lattice[j][k] for k in range(n)):
                adj_matrix[i][j] = 1
                adj_matrix[j][i] = 1

    max_antichain_size = 0
    visited = [False] * n
    stack = []

    def dfs(node):
        nonlocal max_antichain_size
        if visited[node]:
            return
        visited[node] = True
        stack.append(node)

        for neighbor in range(n):
            if adj_matrix[node][neighbor] == 1:
                dfs(neighbor)

        if len(stack) > max_antichain_size:
            max_antichain_size = len(stack)

        stack.pop()
        visited[node] = False

    for node in range(n):
        dfs(node)

    return max_antichain_size

# Function to build the concept lattice
def build_concept_lattice(M, L):
    n = len(L)
    concepts = []
    for i in range(1 << n):
        concept = [0] * n
        for j in range(n):
            if (i >> j) & 1:
                concept[j] = L[j]

```

```

        concepts.append(concept)

    lattice = []
    for i in range(len(concepts)):
        is_antichain = True
        for j in range(i+1, len(concepts)):
            if all(concepts[i][k] <= concepts[j][k] for k in range(n)) and all(concepts[j][k] <= c
                is_antichain = False
                break
        if is_antichain:
            lattice.append(concepts[i])

    return lattice

# Function to run a single trial
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(3, 5)
    M = [random.getrandbits(n) for _ in range(1 << n)]
    L = [(-1 if i < n else 1) * (1 << i) for i in range(2*n)]

    prime_implicants = quine_mccluskey(M)
    sigma_DNF = len(prime_implicants)

    concept_lattice = build_concept_lattice(M, L)
    w_f = antichain_width(concept_lattice)

    if w_f < sigma_DNF:
        return {
            "metric_name": "w(f)/sigma_DNF",
            "metric_value": float(w_f) / sigma_DNF,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"({n}, {M}, {L}) with w(f)={w_f} < sigma_DNF={sigma_DNF}"
        }
    elif w_f > 2 * n * sigma_DNF:
        return {
            "metric_name": "w(f)/sigma_DNF",
            "metric_value": float(w_f) / sigma_DNF,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"({n}, {M}, {L}) with w(f)={w_f} > 2*n*sigma_DNF={2 * n * sigma_DNF}"
        }
    else:
        return {
            "metric_name": "w(f)/sigma_DNF",
            "metric_value": float(w_f) / sigma_DNF,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

```

```

results = []
for seed in seeds:
    trial = run_trial(seed)
    print(f"TRIAL: {trial}")
    results.append(trial)

if all(trial["conjecture_holds"] for trial in results):
    mean_value = sum(trial["metric_value"] for trial in results) / len(results)
    std_value = math.sqrt(sum((trial["metric_value"] - mean_value) ** 2 for trial in results))
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, trial in zip(seeds, results) if not trial["conjecture_holds"])
    counterexample = next(trial["counterexample"] for trial in results if not trial["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_168d9d24.py", line 144, in <module>
    trial = run_trial(seed)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_168d9d24.py", line 106, in run_trial
    prime_implicants = quine_mccluskey([tt[i] << i for i in range(n)])
                        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_168d9d24.py", line 30, in quine_mccluskey
    if pi1 != pi2 and pi1[:-1] == pi2[:-1] and pi1[-1] != pi2[-1]:
        ~~~~~
TypeError: 'int' object is not subscriptable

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a `TypeError` in the `quine_mccluskey` routine before producing any data, so no instances of  $w(f)$  vs  $\sigma\_DNF(f)$  were actually compared. No support or falsification evidence exists. | next: Fix the prime-implicant routine (pass proper bit-pattern strings/tuples, not raw ints) and rerun the full  $n=3,4,5$  sweep, reporting min/max of  $w/\sigma\_DNF$ .

## 11. Audit log (LLM calls)

Total LLM calls: 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 409175       |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 6245         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 3379         |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 6700         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 49519        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15466        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15169        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 21688        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 4877         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 532217 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in `research/audit/6010af3a708c.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6010af3a708c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/6010af3a708c.tar.gz` (if generated)